

Computational Graphic Statics

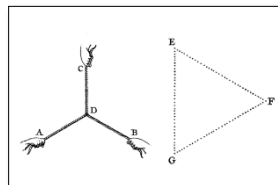
Dr. Lluís Enrique Monzó
Block Research Group (Prof. Dr. Philippe Block)

Week 10

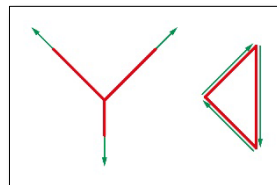
9:45 – 10:30	Recap of last module's exercise Lecture: Algebraic Graphic statics
10:30 – 10:45	Break
10:45 – 11:30	Tutorial: Interactive Graphic Statics
11:30 – 11:45	Break
11:45 – 12:30	Tutorial: Interactive Graphic Statics

Schedule

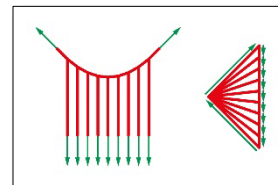
A	Week 1 (20/9)	Lecture	Introduction to graphic statics
		Tutorial	Fundamentals of graphic statics Introduction to Rhino and Grasshopper
		Intro exercise	EX A-1: Graphic statics in Rhino EX A-2: Grasshopper basics
B	Week 2 (27/9)	Submission	EX A-1 and EX A-2
		Lecture	Algorithmic design & thinking
		Tutorial 1	Single node bridge
	Week 3 (04/10)	Intro exercise	EX B-1 : Single node bridge
		Work session	EX B-1
		Week 4 (11/10)	Submission
	Tutorial 2		Form finding of funicular structures
	Intro exercise		EX B-2 : Form finding of funicular structures
	Week 5 (18/10)	Work session	EX B-2
		Week 6: seminar week	
	C	Week 7 (01/11)	Submission
Lecture			Computational Graphic Statics
Tutorial 1			Programming in Grasshopper Python
Week 8 (08/11)		Intro exercise	EX C-1: Programming in Grasshopper Python
		Submission	EX C-1
		Tutorial 2	Form finding of funicular structures in Grasshopper Python
Week 9 (15/11)		Intro exercise	EX C-2: Form finding of funicular structures in Grasshopper Python
		Work session	EX C-2
D	Week 10 (22/11)	Submission	EX C-2
		Lecture	Algebraic Graph Statics (AGS)
		Tutorial 1	Analysis of trusses using Interactive Graphic Statics (IGS2)
	Week 11 (29/11)	Intro exercise	EX D-1: Analysis of trusses using Interactive Graphic Statics (IGS2)
		Submission	EX D-1
		Tutorial 2	Form-finding using Interactive Graphic Statics (IGS2)
	Week 12 (06/12)	Intro exercise	EX D-2: Form-finding using Interactive Graphic Statics (IGS2)
		Work session	EX D-2
	Submission	EX D-2	



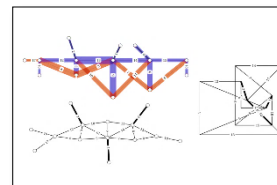
Module I
Introduction to Graphic statics



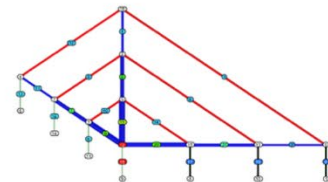
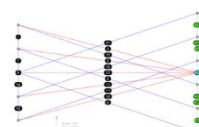
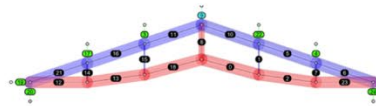
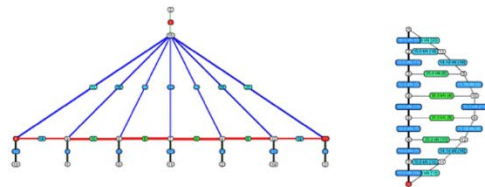
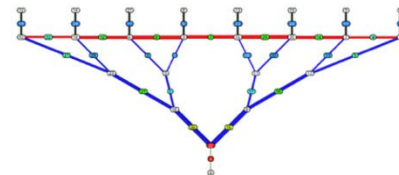
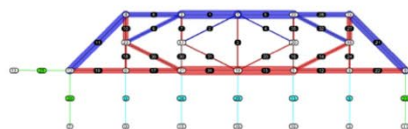
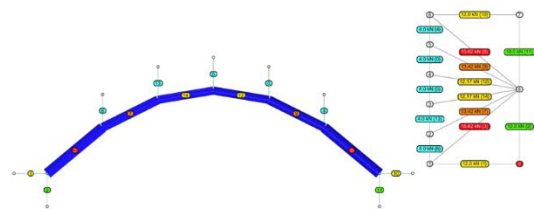
Module II
Procedural Graphic Statics



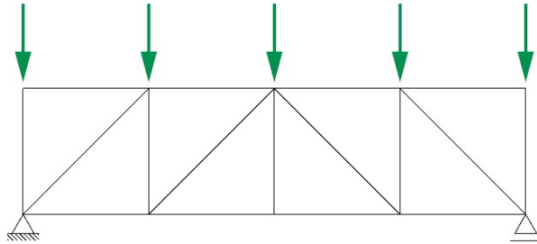
Module III
Programming Graphic Statics



Module IV
Algebraic Graph Statics



Analysis



Input:

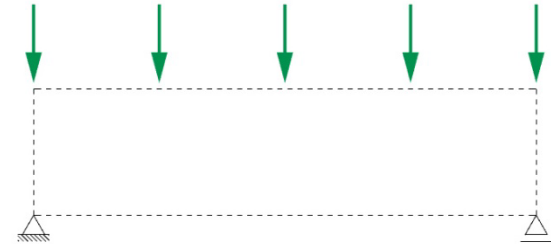
- Loads
- Supports
- Geometry



Trusses
3D trusses



Form-finding

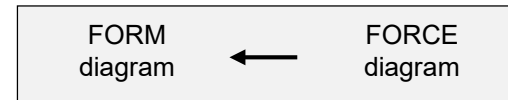


Input:

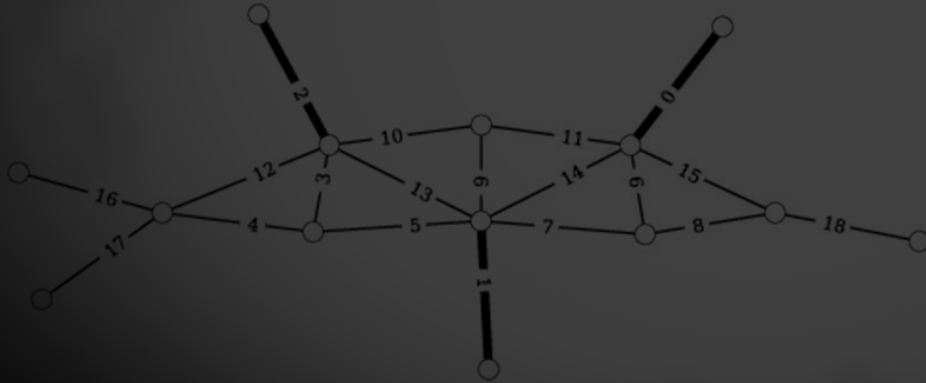
- Loads
- Supports



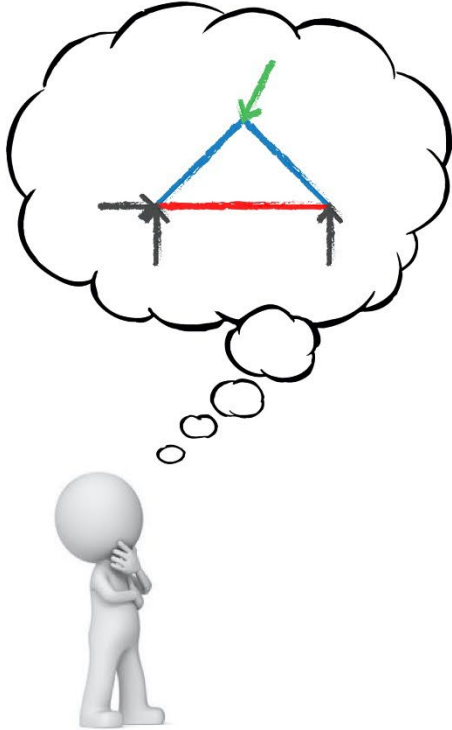
Cables
Arches
Arch-cables
Cable nets
Shells



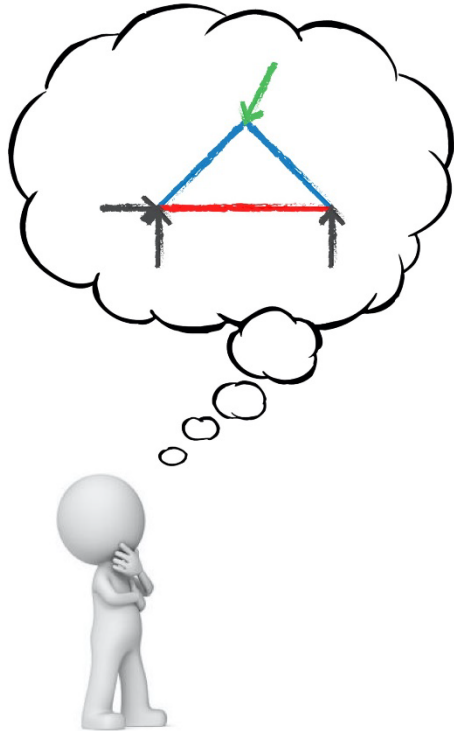
Algebraic Graph(ic) Statics



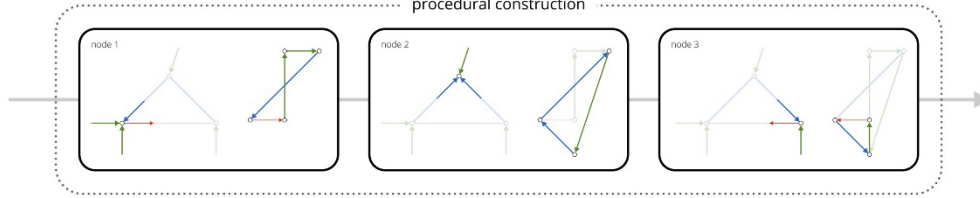
form diagram



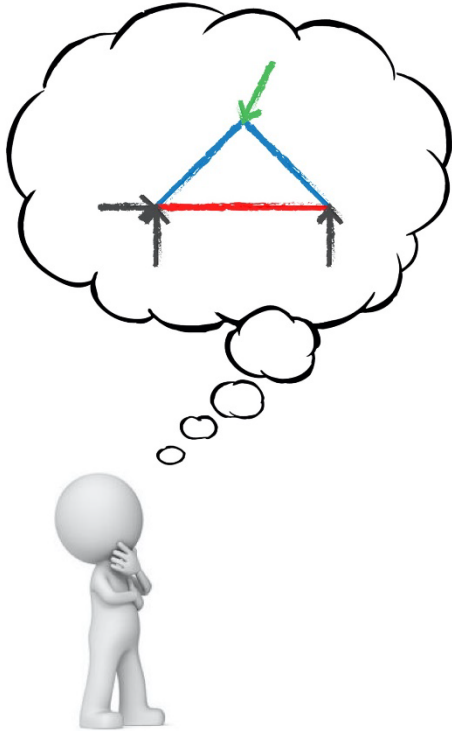
form diagram



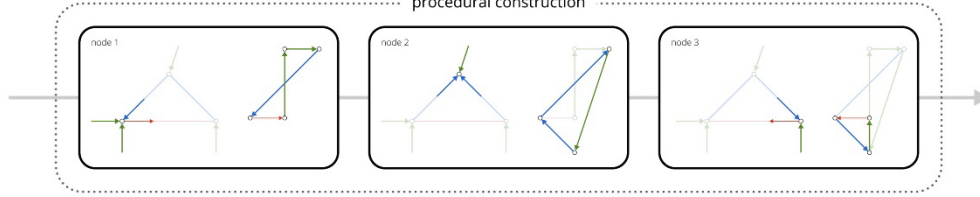
procedural construction



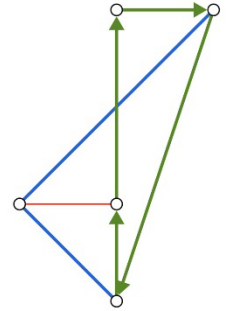
form diagram



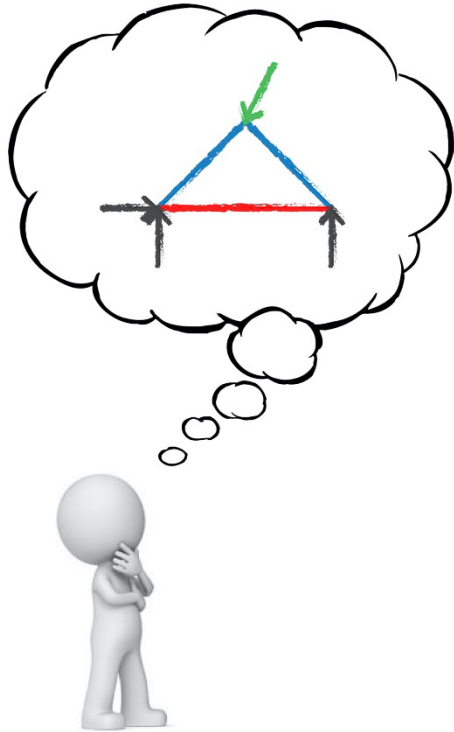
procedural construction



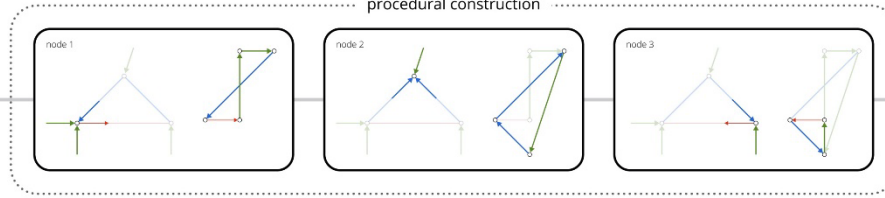
force diagram



form diagram

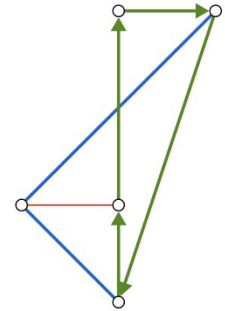


procedural construction

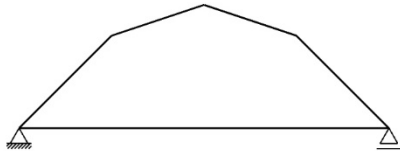
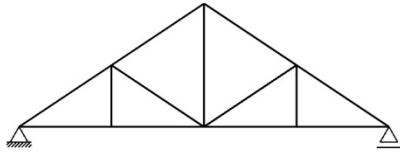


algebraic construction

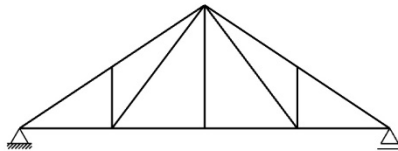
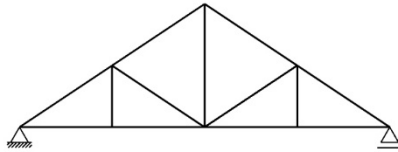
force diagram



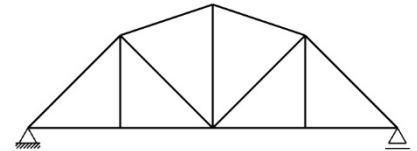
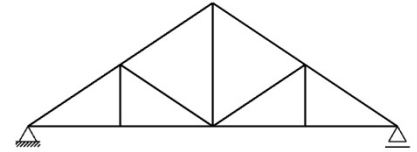
Typology



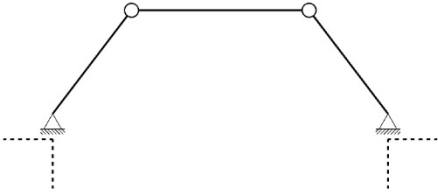
Topology



Geometry



Cables / Arches



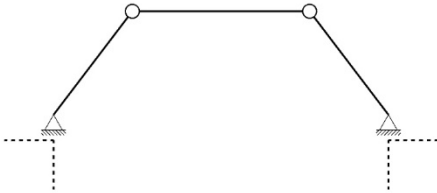
b (bars): 3
r (reactions): 4
n (nodes): 4

$$\boxed{b + r < 2 \cdot n}$$

$$3 + 4 < 2 \cdot 4$$

Unstable!

Cables / Arches



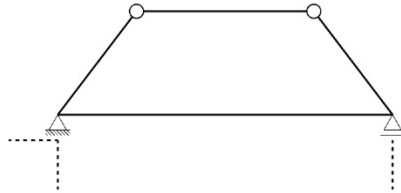
b (bars): 3
r (reactions): 4
n (nodes): 4

$$\boxed{b + r < 2 \cdot n}$$

$$3 + 4 < 2 \cdot 4$$

Unstable!

Arch-cables



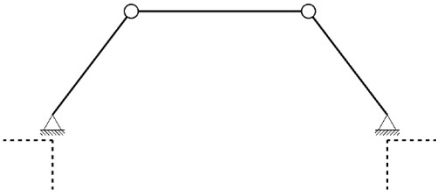
b (bars): 4
r (reactions): 3
n (nodes): 4

$$\boxed{b + r < 2 \cdot n}$$

$$4 + 3 < 2 \cdot 4$$

Unstable!

Cables / Arches



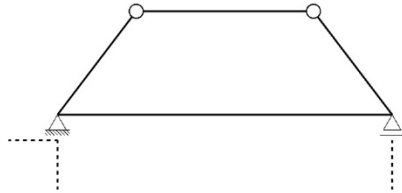
b (bars): 3
r (reactions): 4
n (nodes): 4

$$\boxed{b + r < 2 \cdot n}$$

$$3 + 4 < 2 \cdot 4$$

Unstable!

Arch-cables



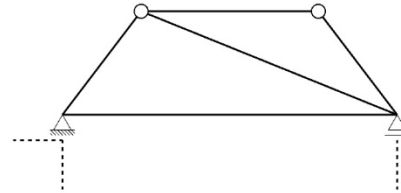
b (bars): 4
r (reactions): 3
n (nodes): 4

$$\boxed{b + r < 2 \cdot n}$$

$$4 + 3 < 2 \cdot 4$$

Unstable!

Trusses

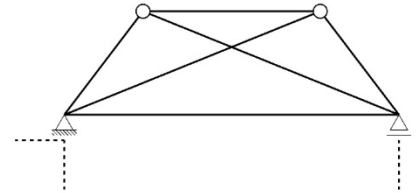


b (bars): 5
r (reactions): 3
n (nodes): 4

$$\boxed{b + r = 2 \cdot n}$$

$$5 + 3 = 2 \cdot 4$$

Stable!
 and
 statically determinate!



b (bars): 6
r (reactions): 3
n (nodes): 4

$$\boxed{b + r > 2 \cdot n}$$

$$6 + 3 > 2 \cdot 4$$

Stable!
 but
 statically indeterminate!

1. Form graph

Connectivity (edges & vertices)

Directed form graph

2. Force graph

Dual graph

Directed force graph

3. Assign forces

4. Solve equilibrium matrix

5. Update force graph

6. Reciprocal form & force graphs

1. Form graph

Connectivity (edges & vertices)

Directed form graph

2. Force graph

Dual graph

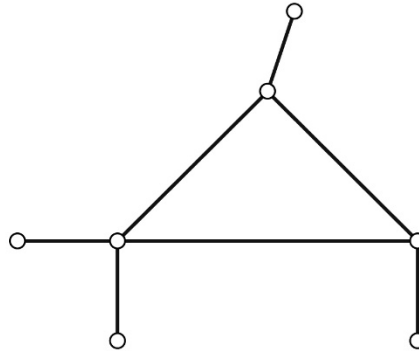
Directed force graph

3. Assign forces

4. Solve equilibrium matrix

5. Update force graph

6. Reciprocal form & force graphs



1. Form graph

Connectivity (edges & vertices)

Directed form graph

2. Force graph

Dual graph

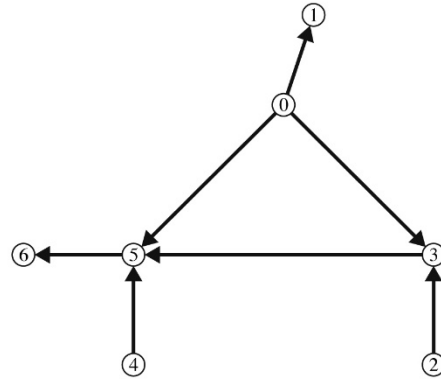
Directed force graph

3. Assign forces

4. Solve equilibrium matrix

5. Update force graph

6. Reciprocal form & force graphs



1. Form graph

Connectivity (edges & vertices)

Directed form graph

2. Force graph

Dual graph

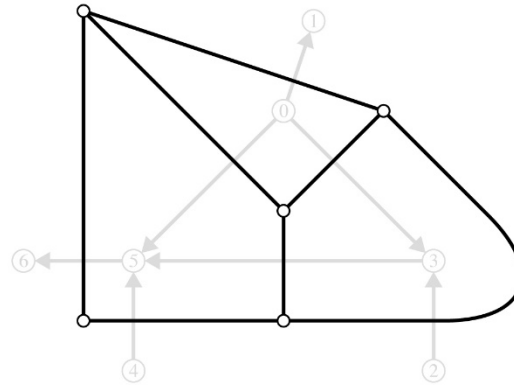
Directed force graph

3. Assign forces

4. Solve equilibrium matrix

5. Update force graph

6. Reciprocal form & force graphs



1. Form graph

Connectivity (edges & vertices)

Directed form graph

2. Force graph

Dual graph

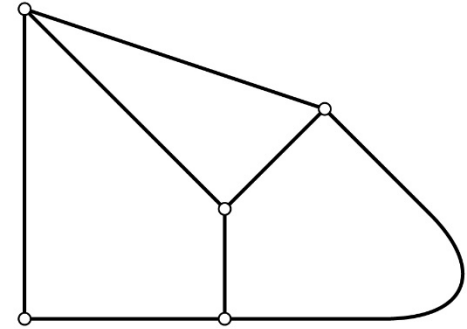
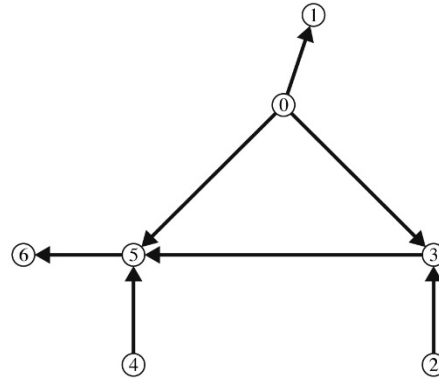
Directed force graph

3. Assign forces

4. Solve equilibrium matrix

5. Update force graph

6. Reciprocal form & force graphs



1. Form graph

Connectivity (edges & vertices)

Directed form graph

2. Force graph

[Dual graph](#)

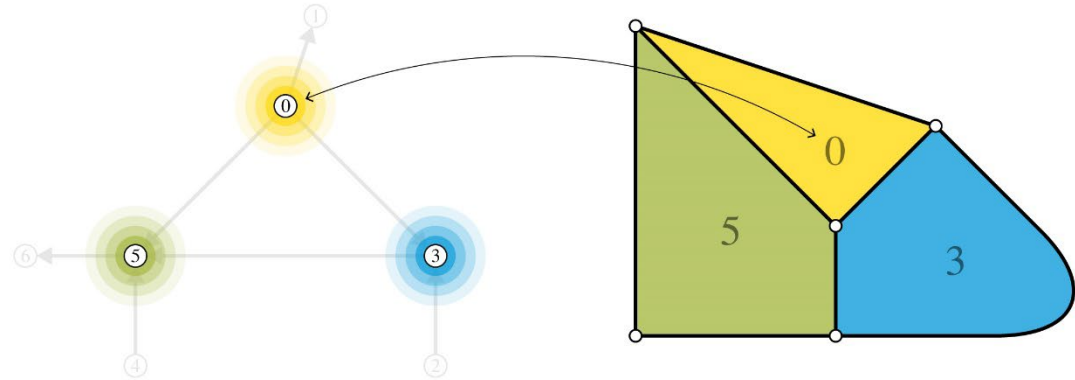
Directed force graph

3. Assign forces

4. Solve equilibrium matrix

5. Update force graph

6. Reciprocal form & force graphs



1. Form graph

Connectivity (edges & vertices)

Directed form graph

2. Force graph

Dual graph

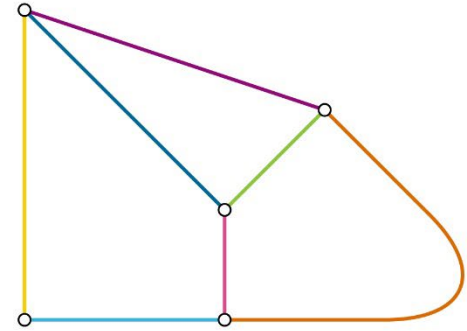
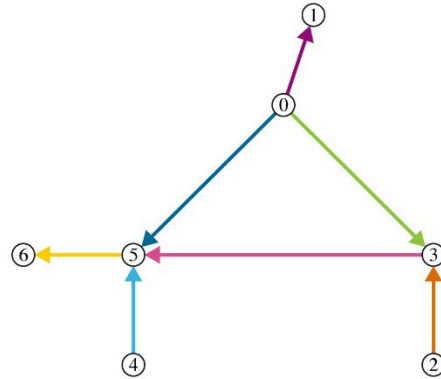
Directed force graph

3. Assign forces

4. Solve equilibrium matrix

5. Update force graph

6. Reciprocal form & force graphs



1. Form graph

Connectivity (edges & vertices)

Directed form graph

2. Force graph

Dual graph

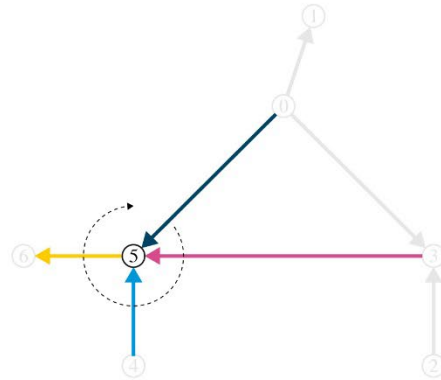
Directed force graph

3. Assign forces

4. Solve equilibrium matrix

5. Update force graph

6. Reciprocal form & force graphs



1. Form graph

Connectivity (edges & vertices)

Directed form graph

2. Force graph

Dual graph

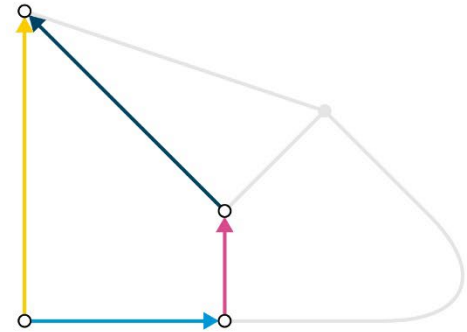
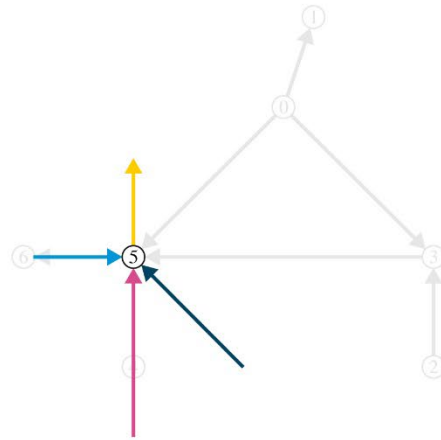
Directed force graph

3. Assign forces

4. Solve equilibrium matrix

5. Update force graph

6. Reciprocal form & force graphs



1. Form graph

Connectivity (edges & vertices)

Directed form graph

2. Force graph

Dual graph

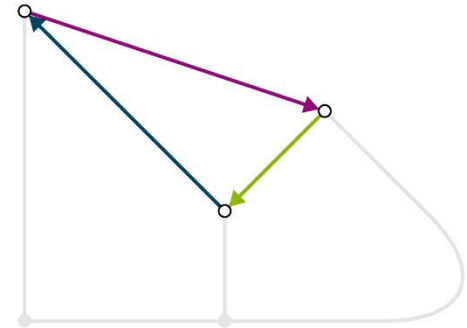
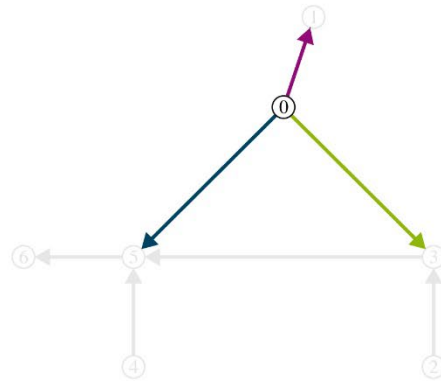
Directed force graph

3. Assign forces

4. Solve equilibrium matrix

5. Update force graph

6. Reciprocal form & force graphs



1. Form graph

Connectivity (edges & vertices)

Directed form graph

2. Force graph

Dual graph

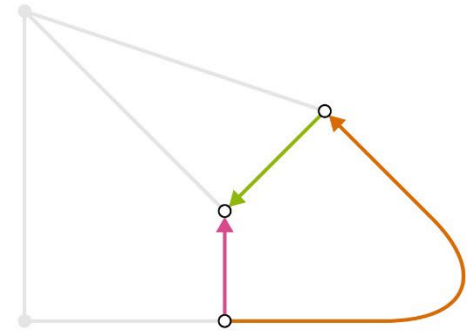
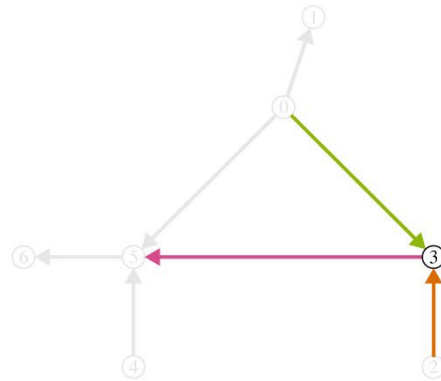
[Directed force graph](#)

3. Assign forces

4. Solve equilibrium matrix

5. Update force graph

6. Reciprocal form & force graphs



1. Form graph

Connectivity (edges & vertices)

Directed form graph

2. Force graph

Dual graph

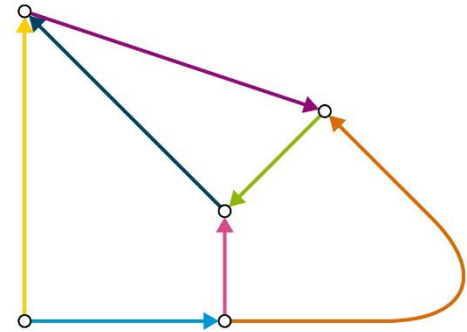
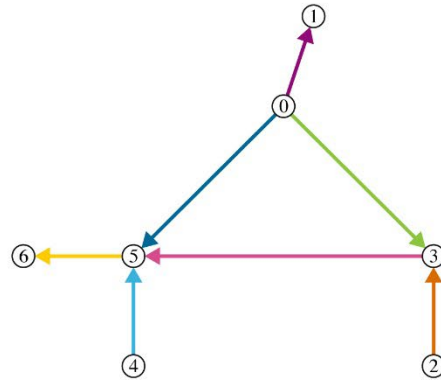
Directed force graph

3. Assign forces

4. Solve equilibrium matrix

5. Update force graph

6. Reciprocal form & force graphs



1. Form graph

Connectivity (edges & vertices)

Directed form graph

2. Force graph

Dual graph

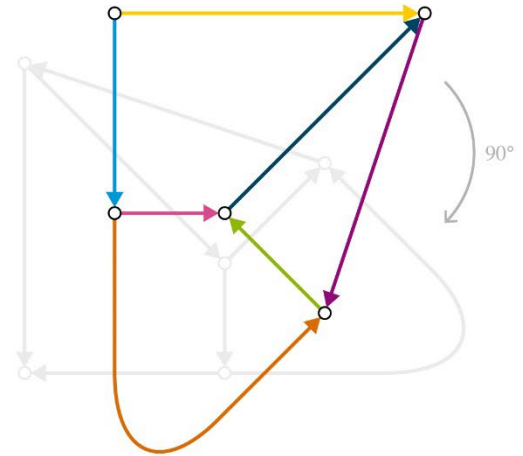
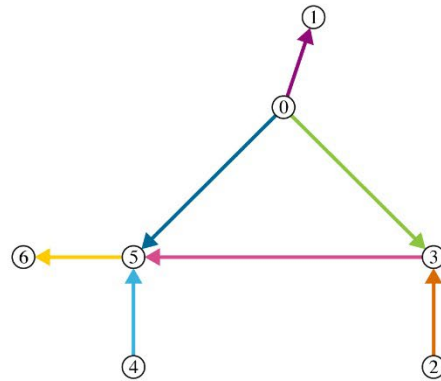
Directed force graph

3. Assign forces

4. Solve equilibrium matrix

5. Update force graph

6. Reciprocal form & force graphs



1. Form graph

Connectivity (edges & vertices)

Directed form graph

2. Force graph

Dual graph

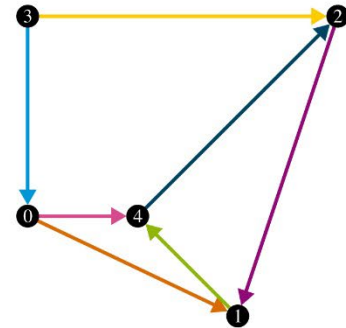
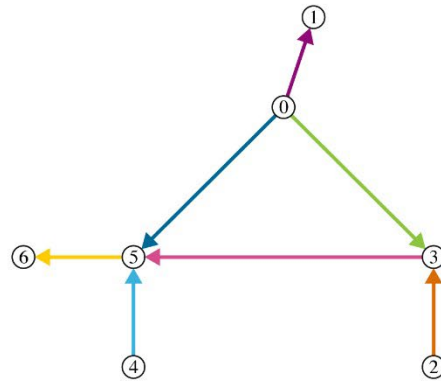
Directed force graph

3. Assign forces

4. Solve equilibrium matrix

5. Update force graph

6. Reciprocal form & force graphs



1. Form graph

Connectivity (edges & vertices)

Directed form graph

2. Force graph

Dual graph

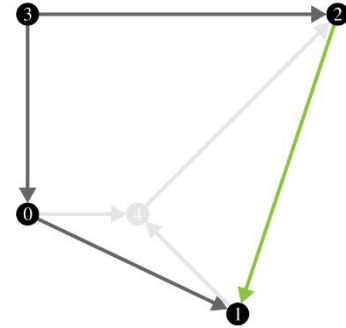
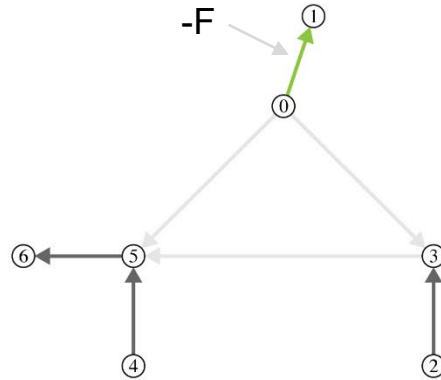
Directed force graph

3. Assign forces

4. Solve equilibrium matrix

5. Update force graph

6. Reciprocal form & force graphs



1. Form graph

Connectivity (edges & vertices)

Directed form graph

2. Force graph

Dual graph

Directed force graph

3. Assign forces

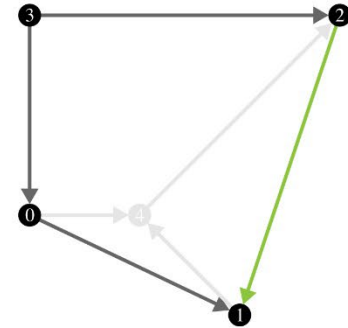
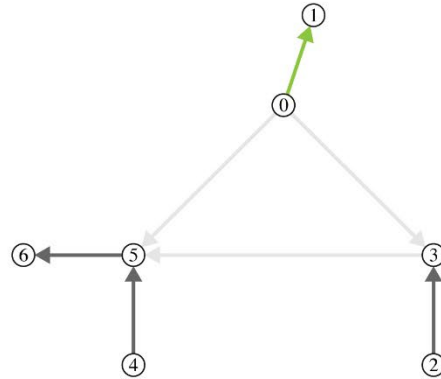
4. Solve equilibrium matrix

5. Update force graph

6. Reciprocal form & force graphs

$$C_{ij} = \begin{cases} +1 & \text{if vertex } i \text{ is the head of edge } j \\ -1 & \text{if vertex } i \text{ is the tail of edge } j \\ 0 & \text{otherwise} \end{cases}$$

$$C_{ij}^* = \begin{cases} +1 & \text{if the face cycle traverses} \\ & \text{edge } j \text{ in the same direction} \\ & \text{as its orientation} \\ -1 & \text{if in the opposite direction} \\ 0 & \text{otherwise} \end{cases}$$



Closed polygon constraint

$$\begin{cases} C_i u^* = 0 \\ C_i v^* = 0 \end{cases}$$

Parallel constraint

$$\begin{cases} u^* = Qu \\ v^* = Qv \end{cases}$$

$$\begin{cases} C_i Uq = 0 \\ C_i Vq = 0 \end{cases}$$

equilibrium matrix

$$Aq = 0, \quad A = \begin{bmatrix} C_i U \\ C_i V \end{bmatrix}$$

1. Form graph

Connectivity (edges & vertices)

Directed form graph

2. Force graph

Dual graph

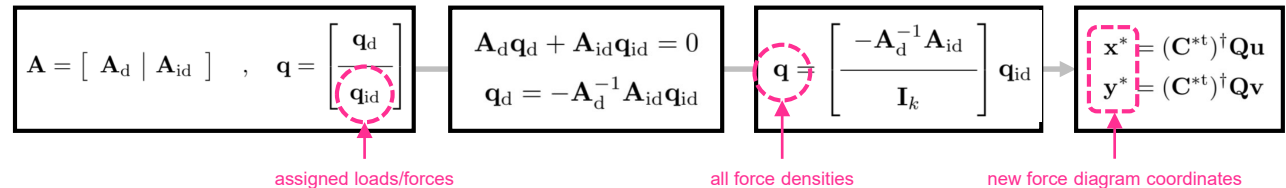
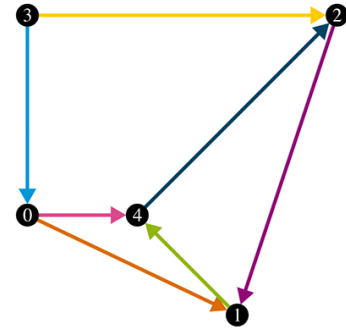
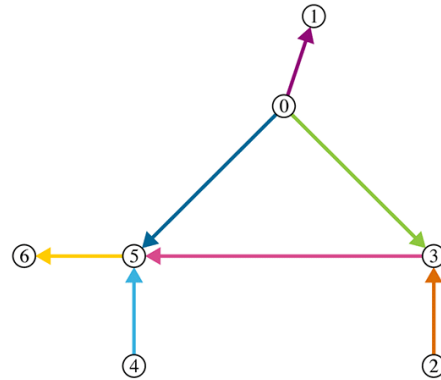
Directed force graph

3. Assign forces

4. Solve equilibrium matrix

5. Update force graph

6. Reciprocal form & force graphs



1. Form graph

Connectivity (edges & vertices)

Directed form graph

2. Force graph

Dual graph

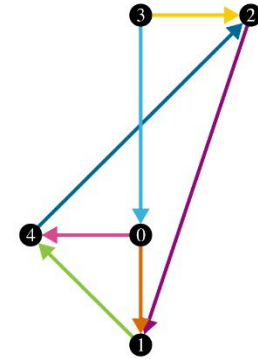
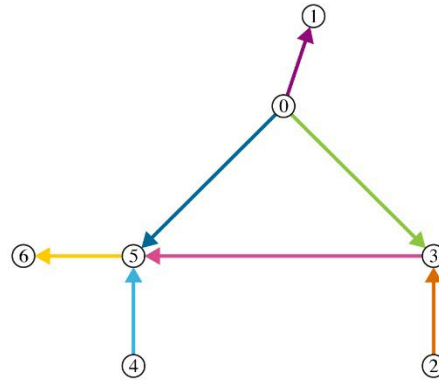
Directed force graph

3. Assign forces

4. Solve equilibrium matrix

5. Update force graph

6. Reciprocal form & force graphs



1. Form graph

Connectivity (edges & vertices)

Directed form graph

2. Force graph

Dual graph

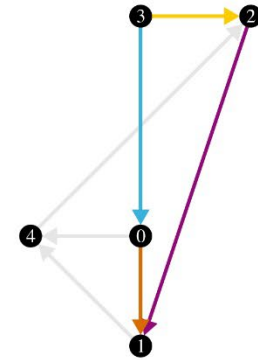
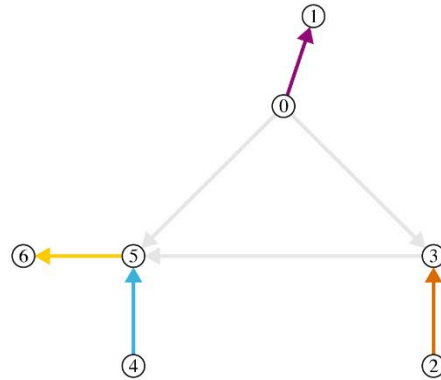
Directed force graph

3. Assign forces

4. Solve equilibrium matrix

5. Update force graph

6. Reciprocal form & force graphs



1. Form graph

Connectivity (edges & vertices)

Directed form graph

2. Force graph

Dual graph

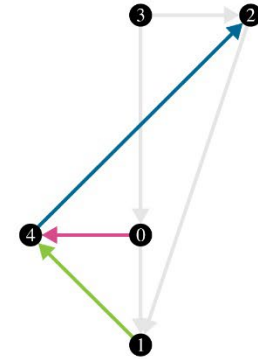
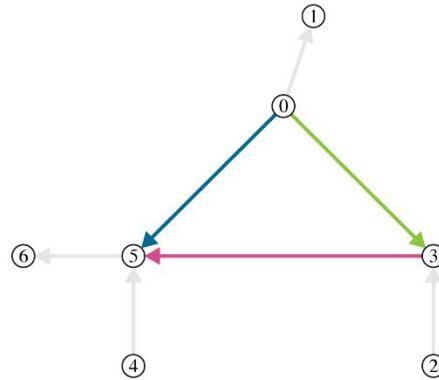
Directed force graph

3. Assign forces

4. Solve equilibrium matrix

5. Update force graph

6. Reciprocal form & force graphs



1. Form graph

Connectivity (edges & vertices)

Directed form graph

2. Force graph

Dual graph

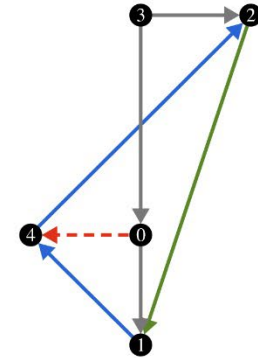
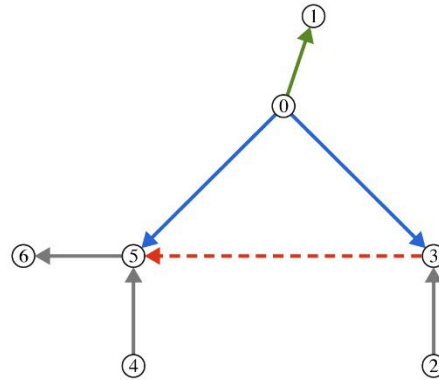
Directed force graph

3. Assign forces

4. Solve equilibrium matrix

5. Update force graph

6. Reciprocal form & force graphs



1. Form graph

Connectivity (edges & vertices)

Directed form graph

2. Force graph

Dual graph

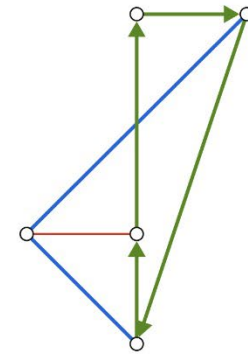
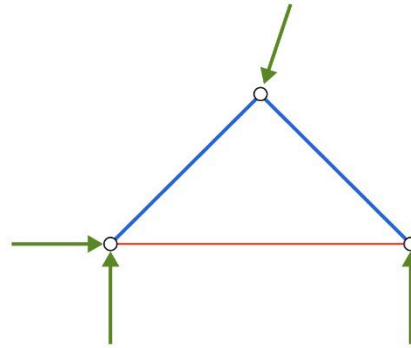
Directed force graph

3. Assign forces

4. Solve equilibrium matrix

5. Update force graph

6. Reciprocal form & force graphs



1. Form graph

Connectivity (edges & vertices)

Directed form graph

2. Force graph

Dual graph

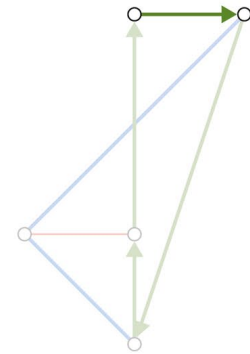
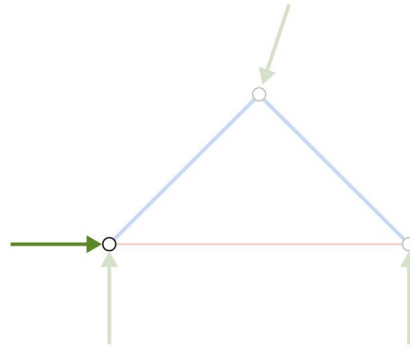
Directed force graph

3. Assign forces

4. Solve equilibrium matrix

5. Update force graph

6. Reciprocal form & force graphs



1. Form graph

Connectivity (edges & vertices)

Directed form graph

2. Force graph

Dual graph

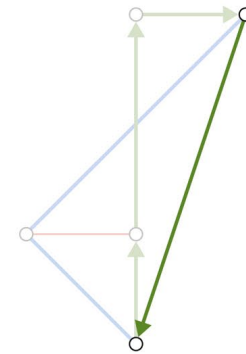
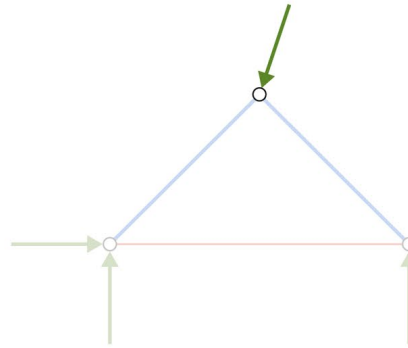
Directed force graph

3. Assign forces

4. Solve equilibrium matrix

5. Update force graph

6. Reciprocal form & force graphs



1. Form graph

Connectivity (edges & vertices)

Directed form graph

2. Force graph

Dual graph

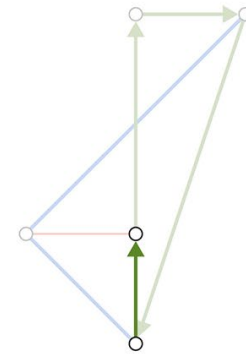
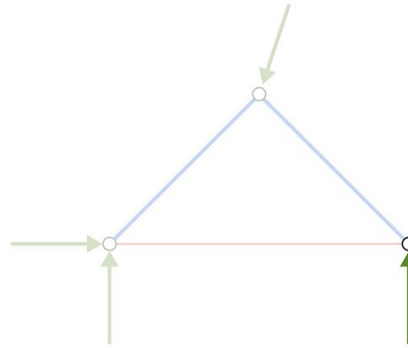
Directed force graph

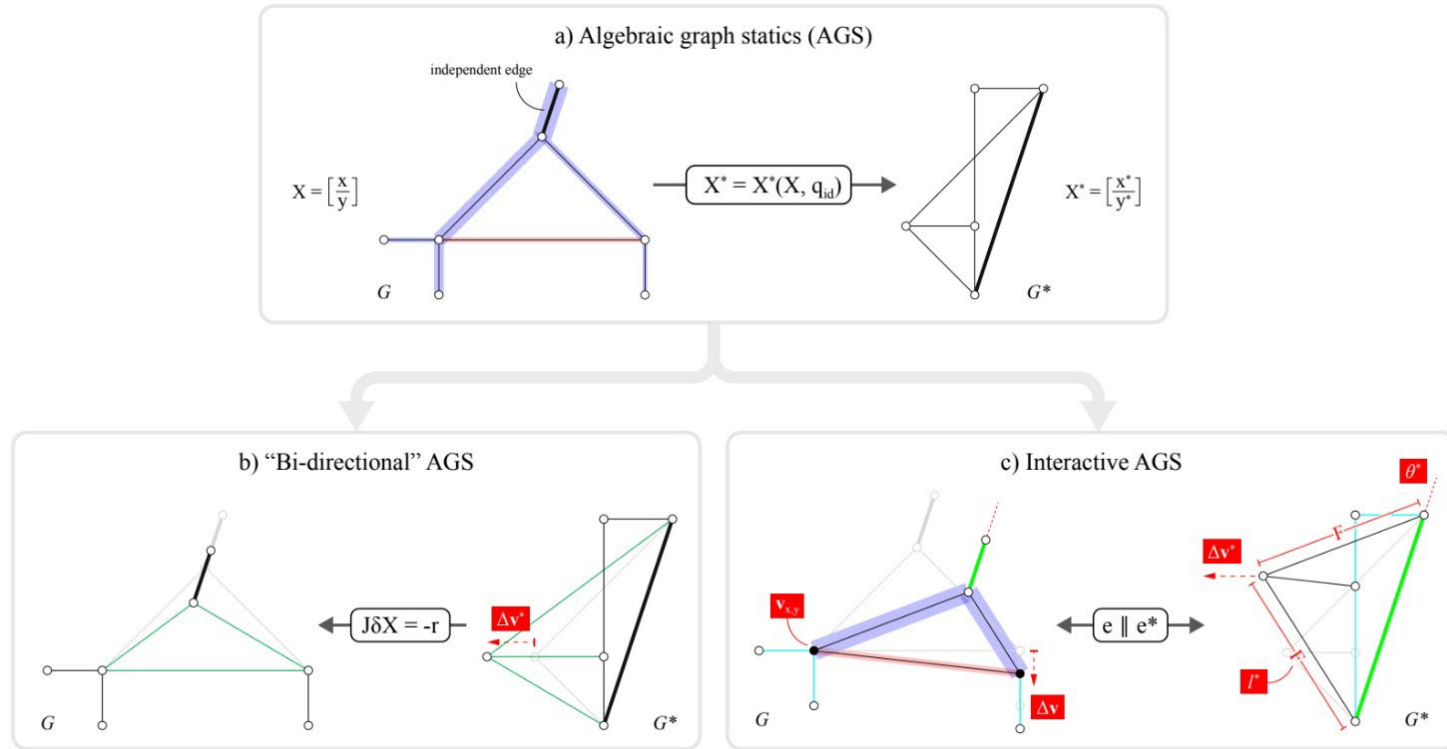
3. Assign forces

4. Solve equilibrium matrix

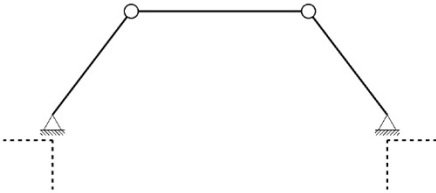
5. Update force graph

6. Reciprocal form & force graphs





Cables / Arches



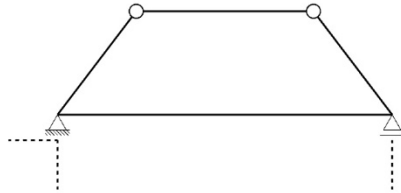
b (bars): 3
r (reactions): 4
n (nodes): 4

$$\boxed{b + r < 2 \cdot n}$$

$$3 + 4 < 2 \cdot 4$$

Unstable!

Arch-cables



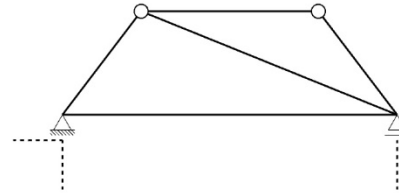
b (bars): 4
r (reactions): 3
n (nodes): 4

$$\boxed{b + r < 2 \cdot n}$$

$$4 + 3 < 2 \cdot 4$$

Unstable!

Trusses

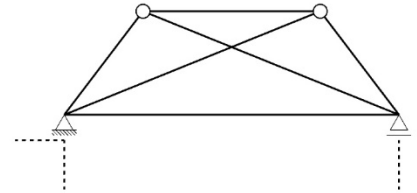


b (bars): 5
r (reactions): 3
n (nodes): 4

$$\boxed{b + r = 2 \cdot n}$$

$$5 + 3 = 2 \cdot 4$$

Stable!
 and
 statically determinate!



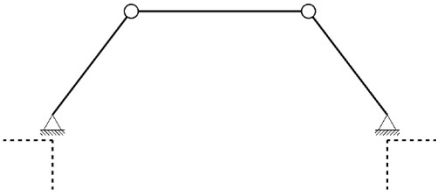
b (bars): 6
r (reactions): 3
n (nodes): 4

$$\boxed{b + r > 2 \cdot n}$$

$$6 + 3 > 2 \cdot 4$$

Stable!
 but
 statically indeterminate!

Cables / Arches



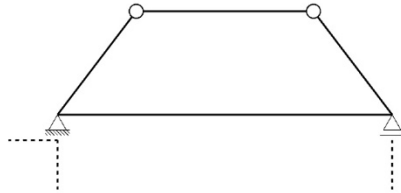
b (bars): 3
r (reactions): 4
n (nodes): 4

$$\boxed{b + r < 2 \cdot n}$$

$$3 + 4 < 2 \cdot 4$$

Unstable!

Arch-cables



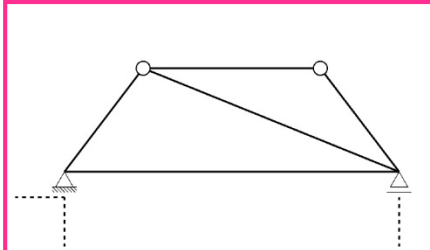
b (bars): 4
r (reactions): 3
n (nodes): 4

$$\boxed{b + r < 2 \cdot n}$$

$$4 + 3 < 2 \cdot 4$$

Unstable!

Trusses

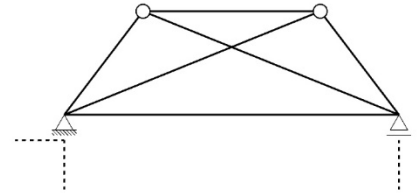


b (bars): 5
r (reactions): 3
n (nodes): 4

$$\boxed{b + r = 2 \cdot n}$$

$$5 + 3 = 2 \cdot 4$$

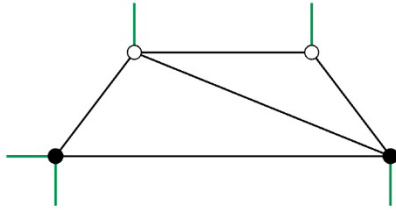
Stable!
 and
 statically determinate!



b (bars): 6
r (reactions): 3
n (nodes): 4

$$\boxed{b + r > 2 \cdot n}$$

6 + 3 > 2 \cdot 4
 Stable!
 but
 statically indeterminate!

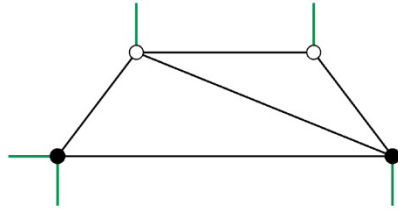


e (edges): 10

n (nodes): 4

$$e - 2 \cdot n = 10 - 2 \cdot 4 = 2$$

We must define two
forces in the system!

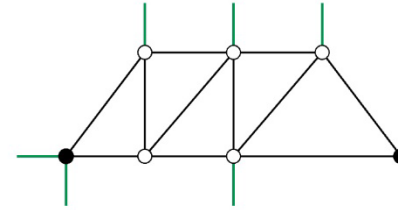


e (edges): 10

n (nodes): 4

$$e - 2 \cdot n = 10 - 2 \cdot 4 = 2$$

We must define two
forces in the system!

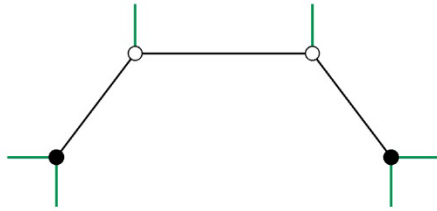


e (edges): 18

n (nodes): 7

$$e - 2 \cdot n = 18 - 2 \cdot 7 = 4$$

We must define four
forces in the system!

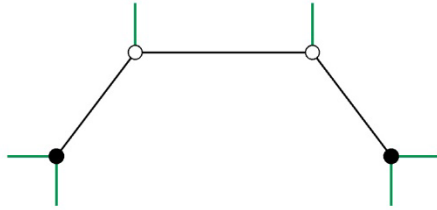


e (edges): 9

n (nodes): 4

$$e - 2 \cdot n = 9 - 2 \cdot 4 = 1$$

We must define one
force in the system!

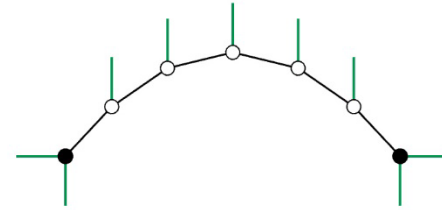


e (edges): 9

n (nodes): 4

$$e - 2 \cdot n = 9 - 2 \cdot 4 = 1$$

We must define one
force in the system!



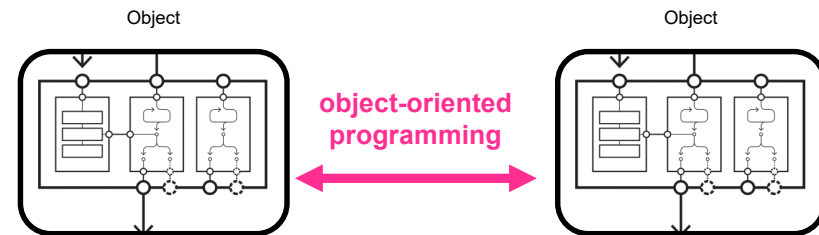
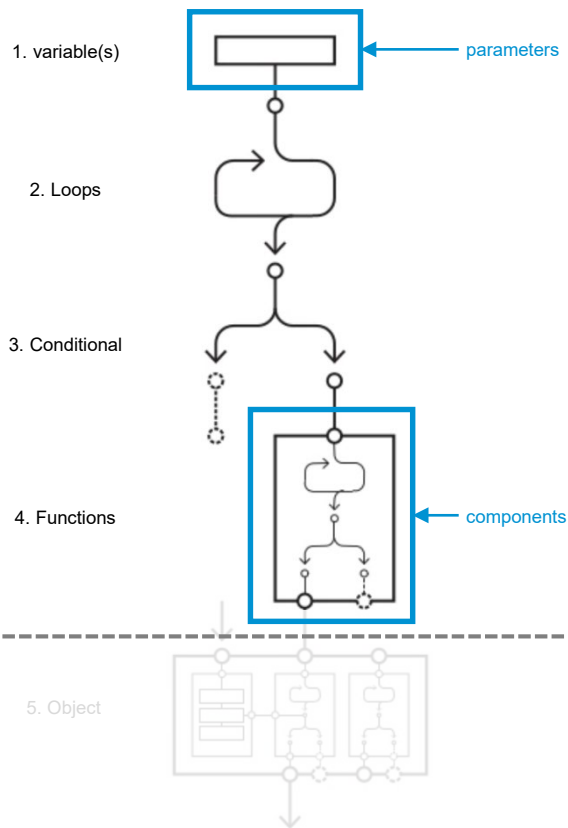
e (edges): 15

n (nodes): 7

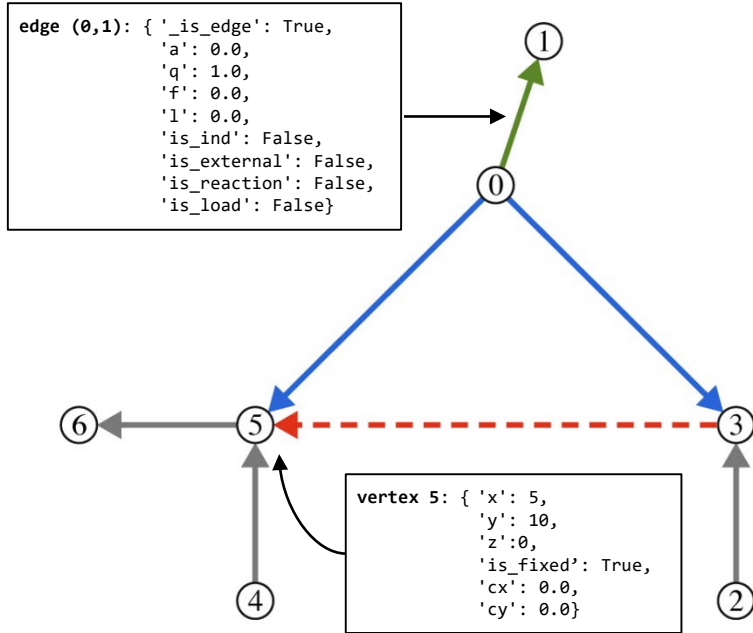
$$e - 2 \cdot n = 15 - 2 \cdot 7 = 1$$

We must define one
force in the system!

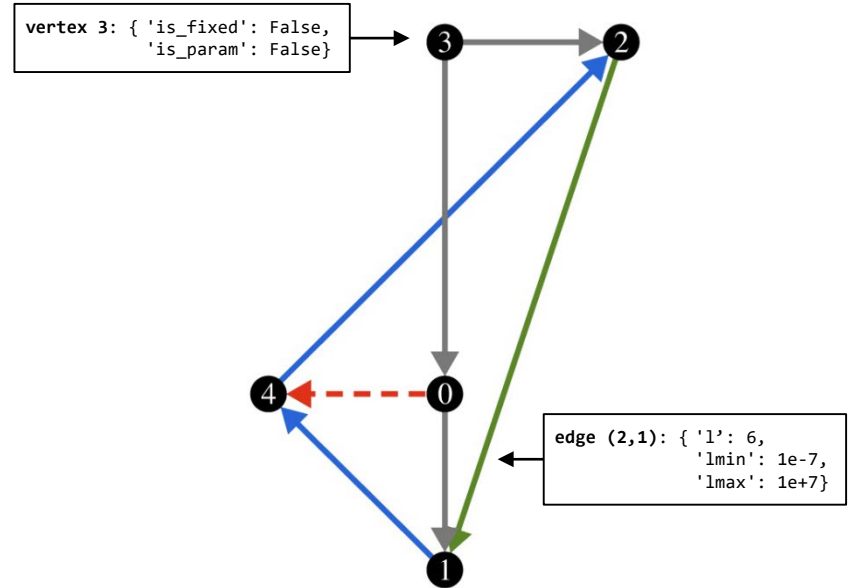
**linear
programming**
(Grasshopper)



form data



force data





COMPAS

Open-source, Python-based framework for computational research and collaboration
in architecture, engineering and digital fabrication

```
def smooth_mesh_length(mesh, lmin, lmax, fixed=None, callback=None):
    """Smooth the mesh length.

    Parameters
    ----------
    mesh : trimesh.Trimesh
        The mesh to smooth.
    lmin : float
        The minimum length of the edges.
    lmax : float
        The maximum length of the edges.
    fixed : list of str
        The names of the edges that are fixed.
    callback : callable
        A callback function that is called for each edge.

    Returns
    -------
    mesh : trimesh.Trimesh
        The smoothed mesh.
    """
    # Get the edges of the mesh.
    edges = mesh.edges_unique

    # Get the length of each edge.
    lengths = mesh.edges_unique_lengths

    # Sort the edges by length.
    sorted_edges = sorted(zip(edges, lengths))

    # Iterate over the edges.
    for key, length in sorted_edges:
        # If the edge is fixed, skip it.
        if key in fixed:
            continue

        # If the edge is not fixed, smooth it.
        if not fixed:
            smooth_edge(mesh, key, lmin, lmax)

        # If a callback is provided, call it.
        if callback:
            callback(mesh, key, callback_args)
```